Door Alarm Security System using Arduino in TinkerCad

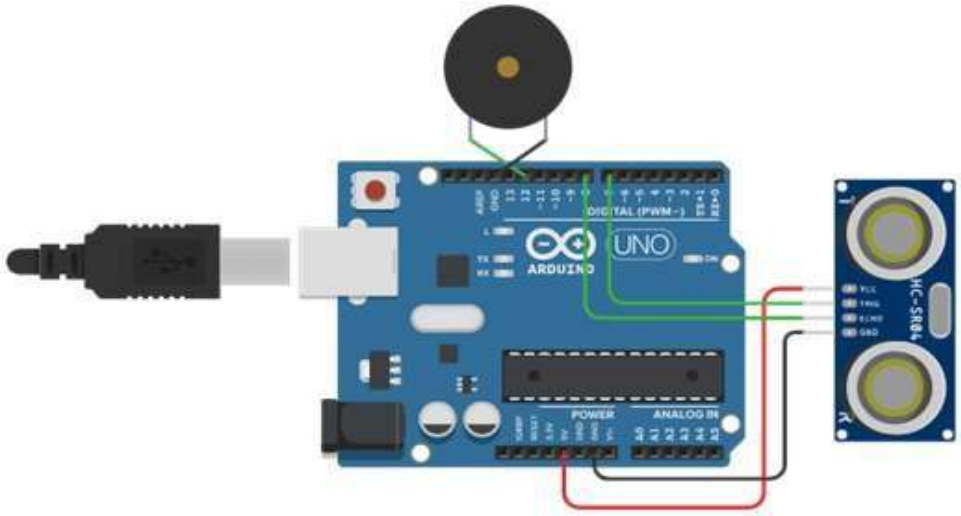
(By Sankeeth Kurian)

TinkerCad is an online 3D modeling program that runs in a web browser. The Door Alarm Security System using Arduino in TinkerCad is a simple electronics project designed to create a security system for a door using an Arduino microcontroller and various electronic components. The project aims to alert the user whenever the door is opened or closed. It can be a fun and educational project for beginners to learn about Arduino programming, electronic components, and basic circuit design. The Door Alarm Security System using Arduino in TinkerCad is a basic demonstration of how a simple security system can be implemented using an Arduino and various electronic components. It can be further expanded and enhanced to suit specific needs and integrated into more complex security setups. Additionally, using TinkerCad for simulation allows beginners to experiment and learn without needing physical components.

COMPONENTS REQUIRED

1. Arduino board (e.g., Arduino Uno)
2. Ultrasonic distance sensor (HC-SR04)
3. Buzzer (active or passive)
4. Jumper wires

CIRCUIT DIAGRAM



Here's how the project works:

1. Set up the hardware:
   - Place the Arduino board on the TinkerCad workspace.
   - Connect the V_{CC} and GND pins of the ultrasonic sensor to the 5V and GND pins of the Arduino, respectively.
   - Connect the Trig and Echo pins of the ultrasonic sensor to any digital pins of the Arduino, for example, Trig to pin 7, and Echo to pin 8.

- Connect one terminal of the buzzer to a digital pin of the Arduino, for example, pin 12.

2. Write the Arduino code:
   - Use the Arduino IDE or TinkerCad's built-in code editor to write the program.
   - The code will use the ultrasonic sensor to detect the distance of the door from the sensor.
   - When the distance is within a certain threshold (indicating the door is closed), the buzzer will remain silent, and the LED (if used) will be turned off.
   - If the distance exceeds the threshold (indicating the door is open), the buzzer will start sounding an alarm, and the LED (if used) will be turned on to indicate the alarm status.

3. Implement the logic:
   - In the Arduino code, initialize the pins for the ultrasonic sensor and buzzer.
   - Create a loop that reads the distance from the ultrasonic sensor using the Trig and Echo pins.
   - Compare the distance to a predefined threshold value (you can experiment and find a suitable value for your door).
   - If the distance is greater than the threshold, set the buzzer pin to HIGH to activate the alarm.
   - If the distance is within the threshold, set the buzzer pin to LOW to turn off the alarm.

4. Upload the code to the Arduino:
   - Once the code is ready, verify it for any errors and then upload it to the Arduino board via the USB cable.

5. Test the Door Alarm Security System:

- Place the ultrasonic sensor near the door in a way that it can detect the door's position.
- Open and close the door to observe the behavior of the alarm system.
- The buzzer should sound an alarm when the door is opened, and they should stop when the door is closed.

RESULTS

Figure: Normal case (Left) and alarm triggers (Right)

 # CODE

```
int triggerPin = 7; int echoPin = 8;
int buzzerPin = 12; long duration,
distanceInCm, inches;

void setup()
{
  Serial.begin(9600);
  pinMode(triggerPin, OUTPUT);    // Clear the trigger
  pinMode(echoPin,                          INPUT);
pinMode(buzzerPin, OUTPUT);
  // Reads the echo pin, and returns the sound wave travel time in microseconds
}

void loop()
{
  digitalWrite(triggerPin, LOW);
delayMicroseconds(2);
  // Sets the trigger pin to HIGH state for 10
microseconds   digitalWrite(triggerPin, HIGH);
delayMicroseconds(10);   digitalWrite(triggerPin, LOW);

  duration = pulseIn(echoPin, HIGH);
// measure the ping time in cm
  distanceInCm = microsecondsToCentimeters(duration);
  // convert to inches by dividing by 2.54   inches
= (distanceInCm / 2.54);   if (distanceInCm > 1
&& distanceInCm < 335.2)
  {
    digitalWrite(buzzerPin, HIGH);
    delay(100);
  }
else
  {
    digitalWrite(buzzerPin, LOW);
    delay(100);
  }
  delay(100);
}

long microsecondsToCentimeters(long microseconds)
{
  return microseconds / 29 / 2; }
```

The above code is the Arduino program designed to create a door alarm security system using an ultrasonic distance sensor. The ultrasonic sensor is used to measure the distance between the sensor and an object, which in this case is the door. When the distance exceeds a certain threshold, indicating the door is open, the buzzer will sound an alarm.

Code Explanation Step by Step:

Variable Declaration:

- **triggerPin**: This variable holds the Arduino pin number to which the trigger pin of the ultrasonic sensor is connected. It is set to pin 7.
- **echoPin**: This variable holds the Arduino pin number to which the echo pin of the ultrasonic sensor is connected. It is set to pin 8.
- **buzzerPin**: This variable holds the Arduino pin number to which the positive terminal of the buzzer is connected. It is set to pin 12.
- **duration**: This variable is used to store the time taken for the ultrasonic signal to bounce back, which is used to calculate the distance.
- **distanceInCm**: This variable stores the distance measured by the ultrasonic sensor in centimeters.
- **inches**: This variable is used to store the distance measured in inches.

Setup():

- The **setup()** function is called once when the Arduino starts.
- The function initializes the communication with the Serial Monitor at a baud rate of 9600. This is helpful for debugging and displaying information.
- The triggerPin is set as an OUTPUT, as it is used to send a trigger signal to the ultrasonic sensor.
- The echoPin is set as an INPUT, as it is used to receive the echo signal from the ultrasonic sensor.
- The buzzerPin is set as an OUTPUT, as it will be used to control the buzzer.

Loop():

- The **loop()** function runs repeatedly after the **setup()** function is executed.
- First, the trigger pin is set to LOW to clear any existing trigger signal.
- Then, a 2-microsecond delay is added to ensure a clean LOW state for the trigger pin.
- The trigger pin is then set to HIGH for 10 microseconds to send an ultrasonic pulse.
- After that, the trigger pin is set back to LOW to end the pulse.
- The **pulseIn()** function is used to measure the duration it takes for the echo signal to return. It reads the echo pin and returns the time in microseconds.

- The **microsecondsToCentimeters()** function is called to convert the time duration to distance in centimeters using the formula: distance = (time duration / 2) / 29.

Distance Checking and Buzzer Control:

- The code checks if the distanceInCm is greater than 1 (to avoid false readings) and less than 335.2 (an arbitrary threshold value for indicating the door is open).
- If the condition is met, it means the door is open or something is obstructing the sensor, and the buzzer is turned on by setting the buzzerPin to HIGH.
- If the condition is not met, it means the door is closed or the sensor is not detecting any object within the range, and the buzzer is turned off by setting the buzzerPin to LOW.
- A delay of 100 milliseconds is added after turning the buzzer on or off to control the alarm's duration and prevent continuous activation.

microsecondsToCentimeters() Function:

- This function takes the time duration in microseconds as input and returns the distance in centimeters.
- The formula used here is based on the speed of sound in air, which is approximately 343 meters per second (or 34300 centimeters per second).
- The formula **distance = (time duration / 2) / 29** is used to convert the time duration to distance in centimeters.

In summary, the code uses the ultrasonic sensor to measure the distance from the sensor to an object (the door), and if the distance exceeds a certain threshold, it triggers the buzzer to sound an alarm. This simple setup can be used as a basic door alarm security system to detect if the door is opened or closed.